🎓 **Exam Type: Practical**
⏳ **Duration: 6 Hours**

Customer Segmentation

Project Overview

You are working as a Junior Data Scientist at the analytics division of a large Indian e-commerce company similar to Flipkart or Meesho. The marketing team wants to stop sending identical promotional emails to all 500,000+ customers — instead, they want to group customers into meaningful segments so that each segment can receive targeted offers, improving conversion rates and reducing spam.

You are given the Online Retail II dataset from the UCI Machine Learning Repository — a real transactional dataset from a UK-based online retailer. Your task is to engineer RFM (Recency, Frequency, Monetary) features from raw transactions, then apply and compare three unsupervised clustering algorithms: K-Means, Agglomerative Hierarchical Clustering, and DBSCAN — to discover natural customer segments and interpret what each segment means for the business.

📁 **Dataset: Online Retail II (UCI ML Repository / Kaggle)**

🔗 UCI Link: https://archive.ics.uci.edu/dataset/502/online+retail+ii

🔗 Kaggle Mirror: https://www.kaggle.com/datasets/mashlyn/online-retail-ii-uci

📊 Rows: ~1 million transactions (2009–2011) | Features: 8 columns (InvoiceNo, StockCode, Description, Quantity, InvoiceDate, UnitPrice, CustomerID, Country)

🎯 Goal: Engineer RFM features per customer, then cluster customers into segments using 3 algorithms

⚠️ Note: Use only UK customers (Country == 'United Kingdom') to reduce noise. Drop rows with null CustomerID.

🎯 Learning Objectives

By completing this exam, students will demonstrate:

- Understanding of Unsupervised Learning and how clustering differs from classification.
- RFM feature engineering from raw transactional data — a core industry skill.
- Implementation and tuning of K-Means Clustering (Elbow Method + Silhouette Score).
- Implementation of Agglomerative Hierarchical Clustering with Dendrogram analysis.
- Implementation of DBSCAN with epsilon and min_samples tuning.
- Cluster evaluation using internal metrics: Silhouette Score, Davies-Bouldin Index, Calinski-Harabasz Index.
- Business interpretation: translating cluster labels into actionable marketing personas.

📂 Step 1: Dataset Loading & Exploratory Data Analysis

1.1 Load & Filter the Dataset

- Load online_retail_II.xlsx (or the CSV version). Print .shape, .info(), and .head(10).
- Filter: keep only rows where Country == 'United Kingdom'.
- Drop all rows where CustomerID is null. Report how many rows remain after filtering.
- Drop rows where Quantity <= 0 (returns/cancellations — InvoiceNo starting with 'C'). Report rows dropped.
- Drop rows where UnitPrice <= 0.
- Create a TotalPrice column: TotalPrice = Quantity × UnitPrice.

1.2 Univariate Analysis

- Plot histograms for: Quantity, UnitPrice, TotalPrice (log-scale the x-axis for readability).
- Plot a countplot of the top 10 countries by order count (before filtering to UK — just for context).
- Plot the number of transactions per month over the dataset's date range. Any seasonal spikes?

1.3 Customer-Level Analysis

- Compute total spend per customer. Plot a histogram of TotalSpend per customer (log-scale).
- Compute order count per customer. Who are the top 10 highest-frequency buyers?
- Report: how many unique customers remain after cleaning?

📌 Key insight to investigate: A small % of customers typically account for a large % of revenue (Pareto principle / 80-20 rule). Does your data confirm this? Compute what % of customers drive 80% of total revenue.

🔧 Step 2: RFM Feature Engineering & Preprocessing

2.1 Build the RFM Table

Compute the following three features per CustomerID:

- Recency: number of days between the customer's most recent purchase and a reference date of 2011-12-31.
- Frequency: total number of unique invoices (orders) placed by the customer.
- Monetary: total spend (sum of TotalPrice) by the customer across all orders.
- Print the resulting rfm_df DataFrame — it should have one row per customer with columns: CustomerID, Recency, Frequency, Monetary.
- Print .describe() on the RFM table. Note the range and skewness of each feature.

2.2 Handle Outliers

- Plot boxplots for Recency, Frequency, and Monetary. Identify extreme outliers.
- Cap outliers using the IQR method: for each column, cap values above Q3 + 3×IQR to that threshold (winsorization — do not drop rows).
- Re-plot boxplots after capping to confirm outliers are handled.

2.3 Transform Skewed Features

- Plot histograms of Recency, Frequency, and Monetary before transformation.
- Apply log1p transformation to Frequency and Monetary (both are typically right-skewed).
- Re-plot histograms after transformation. Are the distributions more symmetrical?

2.4 Scale the Features

- Apply StandardScaler to all three RFM features (Recency, Frequency_log, Monetary_log).
- Store the scaled features as rfm_scaled (a numpy array or DataFrame).
- Print the mean and standard deviation of each scaled feature to confirm scaling worked (mean ≈ 0, std ≈ 1).

⚠️ Important: All three clustering algorithms in this exam are sensitive to feature scale. Never run clustering on raw RFM values — always scale first. Failure to scale is a common exam error.

🔵 Step 3: K-Means Clustering

3.1 Elbow Method — Find Optimal k

- Run KMeans for k = 2, 3, 4, 5, 6, 7, 8, 9, 10 (use init='k-means++', n_init=10, random_state=42).
- For each k, compute and store: Within-Cluster Sum of Squares (inertia_).
- Plot the Elbow Curve (k vs Inertia). Mark the elbow point visually.
- Compute Silhouette Score for each k. Plot k vs Silhouette Score. Which k gives the highest Silhouette Score?
- Based on both plots, choose your optimal k. Justify your choice in 2–3 sentences.

3.2 Train Final K-Means Model

- Train KMeans(n_clusters=k_optimal, init='k-means++', n_init=20, max_iter=500, random_state=42).
- Assign cluster labels back to the original (unscaled) RFM DataFrame as a new column: KMeans_Cluster.

3.3 Visualise K-Means Clusters

- 2D Scatter plot: Recency vs Monetary, coloured by KMeans_Cluster. Add a legend.
- 2D Scatter plot: Frequency vs Monetary, coloured by KMeans_Cluster.
- 3D Scatter plot (matplotlib or plotly): Recency vs Frequency vs Monetary, coloured by cluster. This is the key visual — spend time making it readable.

3.4 Profile the K-Means Clusters

- Compute the mean Recency, Frequency, and Monetary for each cluster (on original unscaled values).
- Present this as a formatted DataFrame — sorted by Monetary (descending).
- Assign a business persona to each cluster. Examples: 'Champions', 'At-Risk Customers', 'Hibernating', 'Loyal Customers', 'New Customers'. Justify each label using the cluster's RFM profile.

💡 India context: Think of clusters in Flipkart/Meesho terms — a 'Champion' cluster has low Recency (bought recently), high Frequency (orders often), high Monetary (spends a lot). These are your VIP customers who deserve loyalty rewards.

🌳 Step 4: Agglomerative Hierarchical Clustering

4.1 Dendrogram Analysis

- Compute a linkage matrix using scipy.cluster.hierarchy.linkage(rfm_scaled, method='ward').
- Plot a Dendrogram using scipy.cluster.hierarchy.dendrogram(). Use truncate_mode='level', p=5 for readability.
- Draw a horizontal cut line on the dendrogram. At which distance threshold does cutting give you 3, 4, or 5 clusters? Report your chosen cut and the resulting number of clusters.

4.2 Train Agglomerative Model

- Train AgglomerativeClustering(n_clusters=n_chosen, linkage='ward').
- Also train with linkage='complete' and linkage='average' using the same n_clusters. Compare Silhouette Scores of all three linkage methods.
- Select the best linkage. Assign cluster labels to the RFM DataFrame as Agg_Cluster.

4.3 Visualise & Profile Hierarchical Clusters

- 2D Scatter plot: Recency vs Monetary, coloured by Agg_Cluster.
- 3D Scatter plot: Recency vs Frequency vs Monetary, coloured by Agg_Cluster.
- Compute cluster-wise RFM means (same as Step 3.4). Assign business personas to each cluster.
- Compare: do the Hierarchical clusters match K-Means clusters? If not, where do they disagree? Write 2–3 sentences.

📌 Note: Hierarchical Clustering with Ward linkage tends to produce compact, equally-sized clusters — similar to K-Means. Complete linkage tends to produce unequal clusters. Average linkage sits in between. Discuss what you observe.

🔴 Step 5: DBSCAN Clustering

5.1 Tune Epsilon (ε) Using k-NN Distance Plot

- Compute k-nearest-neighbour distances for k = MinPts - 1 (start with MinPts = 5).
- Use sklearn's NearestNeighbors(n_neighbors=5).fit(rfm_scaled) and compute distances.
- Sort distances and plot the k-NN distance curve. The 'elbow' of this curve is your optimal ε.
- Identify and report the optimal ε from the plot. Justify your choice in 1–2 sentences.

5.2 Train DBSCAN Model

- Train DBSCAN(eps=optimal_eps, min_samples=5, metric='euclidean').
- Assign cluster labels to the RFM DataFrame as DBSCAN_Cluster. Note: label = -1 means Noise.
- Report: (a) number of clusters found (excluding noise), (b) number of noise points, (c) % of customers classified as noise.

5.3 Tune DBSCAN Hyperparameters

- Run a grid of DBSCAN models across: eps = [0.3, 0.5, 0.7, 1.0, 1.5] and min_samples = [3, 5, 8, 10].
- For each combination, compute Silhouette Score (only on non-noise points). Store results in a DataFrame.
- Plot a heatmap of eps vs min_samples coloured by Silhouette Score.
- Select the best (eps, min_samples) combination. Re-train DBSCAN with these optimal parameters.

5.4 Visualise & Profile DBSCAN Clusters

- 2D Scatter plot: Recency vs Monetary, coloured by DBSCAN_Cluster. Show noise points (label=-1) in grey.
- 3D Scatter plot: Recency vs Frequency vs Monetary, coloured by DBSCAN_Cluster (noise in grey).
- Profile each DBSCAN cluster: compute mean RFM values per cluster (exclude noise). Assign business personas.
- Write 2–3 sentences: What types of customers did DBSCAN identify as 'noise'? What does this mean for the marketing team?

⚠️ DBSCAN tip: On scaled RFM data, DBSCAN often identifies noise points as ultra-high-value VIP customers or extreme outliers — customers so different from everyone else that they form no cluster. These are NOT unimportant; they may be your top 1% spenders!

📊 Step 6: Algorithm Comparison & Business Interpretation

6.1 Internal Metrics Comparison Table

Create a comparison table for all three algorithms showing:

- Algorithm name and final hyperparameters used
- Number of clusters found
- Silhouette Score (higher = better, range: -1 to 1)
- Davies-Bouldin Index (lower = better)
- Calinski-Harabasz Index (higher = better)
- % of points classified as noise (DBSCAN only; 0% for others)

Present as a formatted pandas DataFrame. Based on metrics, rank the three algorithms: which produced the most well-separated, compact clusters?

6.2 Cluster Stability Check for K-Means

- Re-run K-Means 5 times with different random_state values (0, 7, 21, 42, 99).
- For each run, compute the Silhouette Score. Report mean ± std deviation.
- Is your K-Means result stable, or does it vary significantly across runs?

6.3 Business Recommendation

Write a 1-page business recommendation (in your notebook as a Markdown cell) addressed to the Flipkart/Meesho Marketing Director, covering:

- Which clustering algorithm you recommend deploying and why.
- How many customer segments you identified and a name + 1-line description for each.
- One specific marketing action for each segment. Example: 'Champions → Invite to an exclusive Flipkart Plus loyalty programme.' 'At-Risk → Send a 20% discount coupon with a 7-day expiry.'
- What additional data (beyond RFM) would improve segmentation quality in the next iteration?

🚀 Step 7: Pipeline, Deployment & GitHub Submission

7.1 Save the Final Clustering Pipeline

- Save the scaler and best clustering model using joblib:
  - joblib.dump(scaler, 'rfm_scaler.pkl')
  - joblib.dump(best_model, 'customer_segmentation_model.pkl')
- Write a predict_segment() function that accepts a new customer's raw Recency, Frequency, Monetary values and returns their cluster label and persona name.
- Test it on 5 hypothetical new customers. Print their RFM inputs, cluster label, and assigned persona.

7.2 GitHub Submission

- Create a GitHub repository named: customer-segmentation-unsupervised-learning.
- Push the following files:
  - CustomerSegmentation_UnsupervisedLearning.ipynb (fully executed notebook)

- rfm_scaler.pkl (saved StandardScaler)
- customer_segmentation_model.pkl (best clustering model)
- summary_report.md (see below)
- requirements.txt (all libraries: pandas, numpy, matplotlib, seaborn, scikit-learn, scipy, plotly, joblib)
- README.md (project overview, dataset link, how to run, cluster personas)

7.3 Summary Report (summary_report.md)

Write a short report (~400–500 words) covering:

- What was the business problem and dataset?
- How did you engineer RFM features and why did you preprocess them the way you did?
- Which algorithm performed best by internal metrics? Did this match your business intuition?
- Describe each customer segment in 1–2 sentences each.
- What would you do next: additional features, semi-supervised refinement, real-time scoring API?

Marking Scheme (50 Marks)

Total Marks: 50 — distributed across 3 components as follows:

Component	Marks	Deliverable
A. Technical Completion	30 Marks	Completed .ipynb with all clustering tasks built, visualised and functional
B. Recorded Video (Face + Screen)	15 Marks	5–10 min MP4/MOV showing face + screen with verbal explanation of clustering results
C. GitHub Repository + README.md	5 Marks	Public GitHub repo with screenshots, README, video link, commits
TOTAL	**50 Marks**	

Component B: Recorded Video — Face + Screen (15 Marks)

Record a video showing your face (webcam picture-in-picture) and your computer's entire screen simultaneously. Explain your clustering choices, EDA findings, and business interpretations verbally throughout — do not just click without speaking.

Video Requirements:

Requirement	Detail
Format	MP4 or MOV, maximum 500 MB
Face visibility	Webcam picture-in-picture overlay visible throughout the full recording

Requirement	Detail
Screen content	Entire screen (by demonstrating .ipynb and clustering plots)
Duration	5 to 10 minutes
Naming convention	Practical_Exam_YourName_GRID.mp4
Upload location	Google Drive (Anyone with link can view) or YouTube (Unlisted)
Link placement	Paste the video URL in your GitHub README.md under the Video section

Component C: GitHub Repository + README.md (5 Marks)

Create a public GitHub repository specifically for this project. Add all files listed in the Deliverables section. The README.md must include: project title, dataset link, setup instructions (pip install -r requirements.txt), a brief description of each cluster persona, and your video link.

📄 Deliverables

Notebook	CustomerSegmentation_UnsupervisedLearning.ipynb (fully executed, all outputs and plots visible)
Saved Models	rfm_scaler.pkl + customer_segmentation_model.pkl
Summary Report	summary_report.md (algorithm comparison + business personas, ~1 page)
GitHub Repo	All of the above + requirements.txt + README.md + video link

✅ **Submission Checklist:**

- ☐ Notebook runs top-to-bottom without errors
- ☐ RFM table built correctly with one row per customer (Step 2)
- ☐ Elbow curve AND Silhouette plot both shown for K-Means (Step 3.1)
- ☐ Dendrogram plotted with cut line marked (Step 4.1)
- ☐ k-NN distance plot used to select DBSCAN epsilon (Step 5.1)
- ☐ Metrics comparison table present for all 3 algorithms (Step 6.1)
- ☐ Business personas assigned and justified for each cluster
- ☐ GitHub repo link shared with examiner
- ☐ Video URL pasted in README.md and shared with examiner

BRING ON YOUR CODING ATTITUDE

Practical Exam — Unsupervised Learning